

# Fusion or Graphs? A Pre-Registered, Cross-Architecture Study of When Hand-Fusing LLM Decode Chains Beats CUDA-Graph Replay

Animesh Srivastava and Sarthak Tyagi

**Abstract**—Operator fusion is widely assumed to speed up the large language model (LLM) decode path by removing both kernel-launch overhead and intermediate DRAM traffic. We argue that the first of these is already solved in production by CUDA Graphs, so the useful question is narrower: does fusion buy anything *beyond* graph replay, and if so, does it come from bytes or from something else? We present DecodeBench, a fail-closed, pre-registered benchmarking harness that (i) always uses a CUDA-Graph baseline, (ii) splits the fused-versus-graph gap into an analytic byte-time estimate  $B$  and an unexplained residual  $S = (t_{\text{graph}} - t_{\text{fused}}) - B$ , and (iii) produces a go/no-go verdict that needs neither a fused kernel nor a profiler. We pre-registered falsifiable hypotheses on Turing (T4) and tested them, unchanged, on Ada (L4) and Blackwell (RTX PRO 6000). The calibration result, a 17 % wall-clock win for a fused split-KV attention decode attributed mostly to  $S$  rather than to bytes, does not transfer. The same kernel ties the graph baseline on Ada and loses by 27 % on Blackwell, and the ratio  $S/B$  falls monotonically with streaming-multiprocessor count ( $+2.74/+4.73 \rightarrow -0.47/-1.00 \rightarrow -10.09/-3.65$ ), inverting the registered mechanism. The byte argument fails independently: the eliminable fraction is structurally about  $4/D$  of traffic (roughly 3 %, independent of context length) and sits below DRAM-counter resolution on every GPU where counters were available. Four supplementary experiments close the obvious objections: a PyTorch SDPA reference point places our graph baseline within 6.4–9.6 % of production FlashAttention; a tuning sweep of the fused kernel’s one exposed parameter still fails to reach parity on Blackwell; a fresh-process re-run reproduces every cell within 1.1 %; and defeating cache-residency parity is shown to inflate PyTorch’s own kernel by 1.66–1.94 $\times$ . We conclude that CUDA-Graph replay is the only defensible baseline for decode-fusion claims, that fusion economics for short decode kernels are architecture-specific and do not extrapolate in either direction, and that speedups reported against naive launches largely measure launch recovery that graphs already provide.

**Index Terms**—GPU kernel fusion, LLM inference, CUDA Graphs, decode latency, performance modeling, pre-registration, reproducible benchmarking.

## I. INTRODUCTION

EVERY production LLM inference engine we are aware of (vLLM [1], TensorRT-LLM [2], and SGLang [3]) runs CUDA Graphs [4] on the decode path. This is standard practice now, not an exotic optimization. A single graph replay submits an entire kernel chain with one CPU-side launch, collapsing the per-kernel launch overhead that dominates short, memory-bound decode steps.

Artifact, raw data, and pre-registration: <https://github.com/animeshsri14/Decodebench>.

This matters for how fusion work is evaluated. A hand-fused kernel benchmarked against *naive per-kernel stream launches* conflates two very different things: launch recovery, which graphs already deliver without anyone writing fused CUDA, and genuine fusion benefit, which is whatever remains. Papers reporting decode-fusion speedups rarely state which of the two they won. If the baseline is naive launches, a large part of the reported gain may be a term the reader’s production stack has already banked.

The narrower and more useful question is therefore: *beyond graph replay, does fusion save bytes (a real additive gain, since intermediates held in registers or shared memory never reach DRAM), or something else?*

This report answers that question for three decode chains on three GPU generations. The answer is mostly negative, and the pattern of failure is what makes it worth reporting.

## A. Contributions

- **C1: A CUDA-Graph-baselined, three-variant measurement.** Every chain is timed as naive-stream, graph-replayed, and fused, which isolates what fusion adds *over graphs* rather than over a weak baseline.
- **C2: The  $B + S$  decomposition.**  $B$  is analytic (tensor sizes divided by achieved bandwidth) and needs neither a fused kernel nor a profiler, which makes the go/no-go verdict a *pre-implementation* classifier.  $S$  is the explicitly unexplained remainder.
- **C3: A pre-registered, fail-closed protocol,** with frozen gates, dated change control, and honest failures, applied to systems benchmarking [5], [6].
- **C4: A cross-architecture negative result.** The T4-calibrated fused kernel does not transfer,  $S/B$  inverts with SM count, and the byte argument is structurally about  $4/D$  and empirically below counter resolution.

## B. On reporting a refutation

We treat the refutations as the main result, not as a limitation to be buried near the end. The study was designed so that it could falsify its own headline, and it did so twice. Pre-registration v1 claimed that the byte term explained at least 80 % of the attention-chain gap; our own gates refuted it on the calibration GPU. Pre-registration v2 replaced it with the  $B + S$  decomposition and a cross-GPU prediction; the confirmatory runs refuted that as well. We report the sequence rather than a polished result, because the sequence is the finding.

*Scope.* This is a focused case study of three FP16, batch-1 decode chains on a fixed, T4-tuned kernel set. It is not a general decode-fusion benchmark, and it is not a claim about optimally tuned fusion on each architecture.

## II. BACKGROUND

### A. Anatomy of decode traffic

A LLaMA-style decoder layer [7] runs RMSNorm [8]  $\rightarrow$  QKV  $\rightarrow$  RoPE [9]  $\rightarrow$  attention  $\rightarrow$  output projection  $\rightarrow$  RMSNorm  $\rightarrow$  gate/up  $\rightarrow$  SwiGLU [10]  $\rightarrow$  down. At batch 1 every projection degenerates into a GEMV: one activation vector against a full weight matrix. Arithmetic intensity is correspondingly low, and the chain is memory-bandwidth-bound rather than compute-bound, the classic roofline situation [11].

### B. The byte argument and its ceiling

Fusing two stages removes the round trip of the intermediate between them. If that intermediate occupies  $S_b$  bytes, fusion saves roughly  $2S_b/\text{BW}$ : one write plus one read. Only the *inter-kernel* intermediate is affected. Input, output, and weight traffic are untouched, because a fused kernel must still read the same weights and stream the same KV cache.

This bounds the opportunity structurally. For the attention chain the eliminable intermediates are the score and softmax buffers ( $H \times L \times 4$  bytes each), while the irreducible traffic is the KV stream ( $2 \times H \times L \times D \times 2$  bytes). The eliminable fraction is therefore about  $4/D$ , roughly 3% at  $D = 128$ , and importantly *independent of context length*, since growing the KV cache grows numerator and denominator together. Long-context pressure does not improve this argument.

### C. Why CUDA Graphs are the baseline

Graph replay submits a captured chain with a single launch, so the surviving launch term  $\Delta_{\text{launch}}$  on a well-formed chain is small or zero. Any fusion “speedup” measured against naive launches is largely the  $\Delta_{\text{launch}}$  of the naive path, a quantity graphs recover without a line of fused CUDA.

## III. COST MODEL AND DECOMPOSITION

Let  $t_{\text{stream}}$  be the chain under naive per-kernel launches,  $t_{\text{graph}}$  the *same kernels* replayed from a captured CUDA Graph, and  $t_{\text{fused}}$  the hand-fused implementation. All are medians over interleaved trials. We use a two-level decomposition:

$$t_{\text{stream}} - t_{\text{graph}} = \Delta_{\text{launch}}, \quad (1)$$

$$t_{\text{graph}} - t_{\text{fused}} = B + S. \quad (2)$$

Equation (1) is the launch overhead that graphs recover, and is measured directly. Equation (2) is what fusion adds *on top of graph replay*. Fig. 1 shows the arrangement.

The byte term is

$$B = \frac{\text{eliminable bytes}}{\text{BW}_{\text{achieved}}}, \quad \text{BW}_{\text{achieved}} = \frac{\text{total bytes}}{t_{\text{graph}}}. \quad (3)$$

One caveat deserves emphasis:  **$B$  is a proportional estimate, not an upper bound.** The linear-scaling assumption

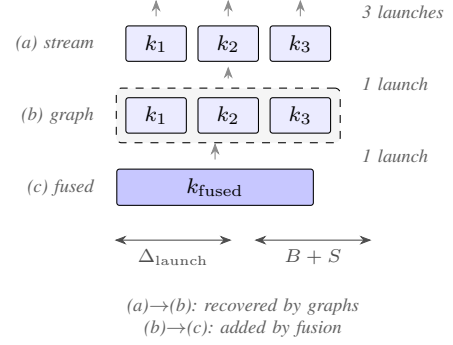


Fig. 1. The three measured variants. Graph replay (b) removes the launch overhead of the naive stream (a) without changing the kernels; only the step from (b) to (c) is attributable to fusion. Reporting a fused kernel against (a) instead of (b) credits fusion with  $\Delta_{\text{launch}}$ , which graphs already recover.

behind (3) fails for launch-, latency-, compute-, or occupancy-bound kernels, for cache-resident traffic, and at serialized stage boundaries. Measured gains can exceed  $B$ , and on T4 they did.

The residual  $S$  absorbs occupancy, register pressure, recompute, cache behaviour, byte-model error, and measurement error together.  **$S$  is a residual, not a mechanism.** A second instrument  $\tau$  (isolated per-kernel durations from Nsight Compute [12]) corroborates only the *sign* of  $S$ , never its magnitude, because profiler replay flushes caches.

Finally, the materiality threshold of 1% classifies chains as *low-byte-opportunity* or *material-byte-opportunity*. We deliberately avoid the labels “launch-bound” and “byte-bound”: the threshold classifies byte opportunity, not the true bottleneck, and a chain below it could still be compute-, latency-, or occupancy-bound.

## IV. METHOD

### A. Workloads

Three chains are measured. **F1** is RMSNorm followed by a GEMV. **F2** is a gate GEMV, an up GEMV, and SwiGLU. **F4** is attention scores, softmax, and the weighted- $V$  product; its fused variant is a split-KV, FlashDecode-style kernel in the spirit of [13], [14]. Geometry is fixed at  $H = 32$  heads,  $D = 128$  head dimension,  $L \in \{2048, 4096\}$ , batch 1, FP16 storage with FP32 accumulation. The per-stage byte model is the one implemented in the released analysis code.

### B. Three variants and residency parity

Weight and KV replicas are sized to defeat L2 across *all three* variants and are rotated identically on every iteration. The rule changed our own results: before residency parity was enforced, fused F1 and F2 appeared to win, but those wins were cache artifacts, and Section VI-B shows the same trap catching a third-party production kernel.

### C. Measurement hygiene

Timing uses CUDA events with an adaptive iteration count so that each trial spans at least 20 ms, with device synchronization at the trial boundaries. Variants are measured in three

rotated interleaved passes of ten trials, so that clock and thermal drift spread across variants instead of loading onto whichever ran last. SM clocks are locked on a best-effort basis, with a recorded warning when the host denies clock control, as it does on vGPU. Each run emits a manifest recording driver, CUDA, and compiler versions, commit hash, clock-lock outcome, variant ordering, and seed, in the spirit of [15], [5].

#### D. Pre-registered, fail-closed gates

The registered gates are: G0 completeness; G1 numerical correctness against a scalar CPU reference ( $\text{abs} < 5 \times 10^{-2}$  OR  $\text{rel} < 2 \times 10^{-2}$ ); G2 GEMV throughput at least 90% of cuBLAS; (a) sign corroboration by  $\tau$ ; (b) analytic-versus-measured DRAM bytes with a counter-resolution floor; (c) a  $\Delta_{\text{launch}}$  noise floor; and (H) the two hypotheses. A warning never counts as a pass.

A check that runs but establishes nothing either way (a  $\tau$  instrument reading inside the  $\pm 5 \mu\text{s}$  band, or a byte delta below counter resolution) is recorded as INDETERMINATE rather than PASS. It contributes no support and does not gate. This accounting change, dated 2026-07-03, moved no threshold and *reduced* our own reported pass counts.

#### E. Uncertainty

We summarize with medians and report fixed-seed bootstrap 95% confidence intervals [16] on  $\Delta_{\text{launch}}$  and on  $S$ ; because  $B$  is linear in  $t_{\text{graph}}$ , uncertainty propagates to  $S$  exactly. Between-run variance is assessed with a second fresh-process run (Section VI-C).

### V. RESULTS

#### A. Headline

Table I gives the top-level outcome and Table II the decomposition. Every value is recomputed from the committed CSVs by the released figure script, not transcribed.

Numerical correctness (G1) is 18/18 on all three parts, and GEMV throughput (G2) is 108.6%, 100.6%, and 101.1% of cuBLAS respectively, so the kernels themselves are sound on every platform.

#### B. T4: calibration

At  $L = 4096$  the gap is  $66.63 \mu\text{s}$ , decomposing as  $B = 11.64$  and  $S = 54.99$ . The win is real and large, but *it is not a byte win*: bytes explain about a sixth of it. The bootstrap interval on  $S$  excludes any value that would change this reading ( $[12.79, 17.91]$  at  $L = 2048$ ;  $[54.76, 57.63]$  at  $L = 4096$ ).

Fused F1 and F2 are *slower* than the graph baseline, by  $33.81$  and  $101.87 \mu\text{s}$  for F1 and by  $14.27$  and  $33.92 \mu\text{s}$  for F2. Hypothesis H1-v2 holds: for launch-bound chains, hand-fusion costs more than it saves once graphs are the baseline.

We note that  $\Delta_{\text{launch}}$  is essentially zero on T4 ( $0.02$  and  $-0.19 \mu\text{s}$ ). On this part the three launches are already free, so graphs buy nothing and the entire fusion gain is  $B + S$ . Section V-E returns to this.

#### C. L4: first refutation

The fused kernel merely ties:  $139.64$  against  $141.90 \mu\text{s}$  at  $L = 2048$ , and  $279.40$  against  $279.39 \mu\text{s}$  at  $L = 4096$ , an exact tie to two decimals.  $S$  is negative at both lengths. Under the registered falsification criterion (the fused kernel losing, or  $S \leq B$ , on any GPU), **H2-v2 is refuted**. The  $\tau$  instrument corroborated three cells and was indeterminate on three, those being small-gap cells inside the  $\pm 5 \mu\text{s}$  band.

Everything else replicated: H1-v2 held, all 18 correctness checks passed, byte totals gated at ratios  $0.87$ – $0.90$ , and the attention byte delta was below counter resolution as predicted. The gates ran unchanged; what failed was the hypothesis.

#### D. Blackwell: second, harder refutation

The fused kernel loses outright, by 27% at  $L = 2048$  and 8% at  $L = 4096$ . This meets the wall-clock falsification criterion directly from the timing grid, with no profiler input at all.

That run is partial for an environmental reason rather than a methodological one. The machine exposes the GPU as an NVIDIA vGPU with MIG enabled; the instance presents 94 SMs rather than the full die, and the host profile blocks performance counters even for a privileged user. All twelve failures are missing-profiler completeness cells. Because  $B$  is analytic,  $S/B$  is still computable from timings alone. A fully gated Blackwell run needs a passthrough machine.

The conditions of H1-v2 also fail here, in the opposite direction: fused F1 *beats* the graph baseline by  $7.78 \mu\text{s}$  at  $L = 4096$ , and fused F2 by  $2.70$  and  $3.09 \mu\text{s}$ , against a  $2 \mu\text{s}$  noise floor. Fusion economics on this part are inverted relative to Turing in both directions: attention fusion hurts, while the launch-bound chains mildly benefit.

#### E. The trend and the registered mechanism

Fig. 2 is the central result.  $S/B$  falls monotonically with SM count at both context lengths and crosses zero between Turing and Ada. The pre-registration predicted the *opposite* slope, on the argument that the  $H=32$ -block softmax stage underutilizes larger GPUs more, so removing it should pay off increasingly. The data run the other way at every step.

A second trend, which we did not register, runs alongside it.  $\Delta_{\text{launch}}$  for the attention chain rises from about zero on T4 to  $0.71/1.85 \mu\text{s}$  on L4 and  $4.22/2.17 \mu\text{s}$  on Blackwell (Table II, Fig. 3). As parts get faster, a fixed launch cost occupies a larger share of a shrinking kernel time. The term graphs recover therefore grows in relative importance exactly as the term fusion adds beyond graphs turns negative.

Chen [17] reports the same direction independently on a very different workload, finding CUDA Graphs worth  $1.259\times$  on H100 but only  $1.028\times$  on L4. Two studies with different methods reach the same conclusion: the balance between launch cost and memory cost is architecture-dependent, and it shifts toward launch on newer hardware.

### VI. SUPPLEMENTARY EXPERIMENTS

The four experiments in this section are *not gated*. No registered threshold was consulted or changed, and the committed

TABLE I

F4 ATTENTION CHAIN: FUSED KERNEL VERSUS THE CUDA-GRAPH BASELINE. THE T4 RESULT IS THE CALIBRATION POINT; L4 AND RTX PRO 6000 ARE CONFIRMATORY RUNS OF THE SAME FROZEN PROTOCOL.

| GPU (architecture)       | SMs | $L=2048$                          | $L=4096$                           | $S/B$ @2048 | $S/B$ @4096 | H1-v2 (F1/F2) |
|--------------------------|-----|-----------------------------------|------------------------------------|-------------|-------------|---------------|
| T4 (Turing)              | 40  | 166.30 vs 187.56 <b>WIN</b>       | 317.53 vs 384.16 <b>WIN (17 %)</b> | +2.74       | +4.73       | holds         |
| L4 (Ada)                 | 58  | 139.64 vs 141.90 <i>tie</i>       | 279.40 vs 279.39 <i>tie</i>        | -0.47       | -1.00       | holds         |
| RTX PRO 6000 (Blackwell) | 94  | 74.72 vs 58.59 <b>LOSS (27 %)</b> | 114.91 vs 106.36 <b>LOSS (8 %)</b> | -10.09      | -3.65       | fails         |

Entries are  $t_{\text{fused}}$  vs  $t_{\text{graph}}$  in  $\mu\text{s}$ ; lower is better. H1-v2 predicts fused F1/F2 never beat the graph baseline; it holds on T4 and L4, and fails on Blackwell, where fused F1 and F2 *win*.

TABLE II

F4 DECOMPOSITION, ALL VALUES IN  $\mu\text{s}$ .  $\Delta_{\text{launch}} = t_{\text{stream}} - t_{\text{graph}}$ ;  
 $\text{GAP} = t_{\text{graph}} - t_{\text{fused}} = B + S$ .

| GPU | $L$  | $t_{\text{stream}}$ | $t_{\text{graph}}$ | $\Delta_{\text{launch}}$ | gap    | $B$   | $S$    |
|-----|------|---------------------|--------------------|--------------------------|--------|-------|--------|
| T4  | 2048 | 187.58              | 187.56             | 0.02                     | +21.26 | 5.68  | +15.58 |
|     | 4096 | 383.97              | 384.16             | -0.19                    | +66.63 | 11.64 | +54.99 |
| L4  | 2048 | 142.61              | 141.90             | 0.71                     | +2.26  | 4.30  | -2.04  |
|     | 4096 | 281.24              | 279.39             | 1.85                     | -0.00  | 8.46  | -8.47  |
| RTX | 2048 | 62.81               | 58.59              | 4.22                     | -16.13 | 1.77  | -17.91 |
|     | 4096 | 108.52              | 106.36             | 2.17                     | -8.55  | 3.22  | -11.77 |

TABLE III

GATE SCORECARD. A FAILURE OF (H) IS A REFUTED HYPOTHESIS, NOT A BROKEN INSTRUMENT; THE RTX FAILURES ARE ALL MISSING-PROFILER COMPLETENESS CELLS.

| GPU | Pass | Indet. | Fail | Overall | Cause of failures    |
|-----|------|--------|------|---------|----------------------|
| T4  | 73   | 3      | 0    | PASS    | none                 |
| L4  | 69   | 5      | 2    | FAIL    | H2-v2 refuted        |
| RTX | 42   | 0      | 12   | FAIL    | G0, profiler blocked |

validation report is unmodified. They narrow stated limitations rather than testing hypotheses.

#### A. A production reference point

The attention comparison is between two hand-written kernels. If both were slow, the decomposition would be measuring our own inefficiency. We therefore timed PyTorch’s `scaled_dot_product_attention`, which dispatches to a FlashAttention backend at this geometry, under the identical protocol and with outputs checked against an FP32 reference at the harness tolerance.

Our graph baseline sits within 6.4 % and 9.6 % of a production kernel (Table IV, Fig. 5), so the Blackwell loss is not an artifact of a weak comparison. The fused kernel is also slower than SDPA, by 38.2 % and 19.7 %, so the refutation no longer depends on any of our own code being fast. Throughout this section, a percentage “slower than” a reference is  $(t - t_{\text{ref}})/t_{\text{ref}}$ , taken against that reference.

One incidental observation: SDPA under a graph and SDPA on a plain stream are indistinguishable (55.22 versus 54.31, and 96.14 versus 96.23, the latter inverted within noise). A single-kernel implementation has no launch overhead for graphs to recover. Launch cost is a property of chain length, not of fusion.

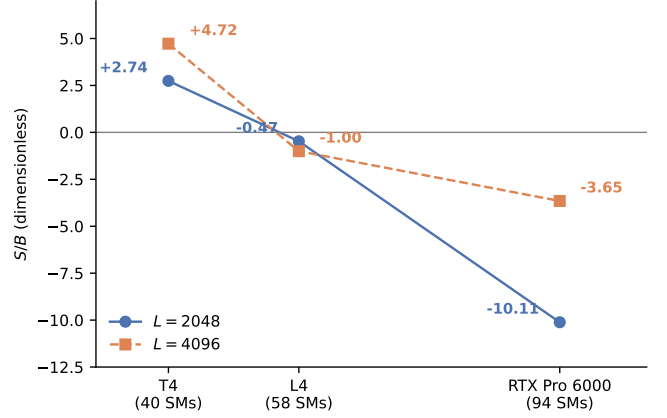


Fig. 2.  $S/B$  against SM count. The registered prediction (dotted) was that this ratio grows with SM count; it falls monotonically and changes sign. Points are recomputed from the committed measurement CSVs.

TABLE IV

BLACKWELL: HAND-WRITTEN VARIANTS AGAINST PYTORCH SDPA, SAME PROTOCOL AND SAME GEOMETRY. VALUES IN  $\mu\text{s}$ .

| $L$  | SDPA (graph) | SDPA (stream) | unfused+graph | fused  |
|------|--------------|---------------|---------------|--------|
| 2048 | 54.31        | 55.22         | 57.99         | 75.07  |
| 4096 | 96.23        | 96.14         | 106.50        | 115.20 |

#### B. The cache-residency trap, on someone else’s kernel

Repeating the measurement with KV pinned to a single buffer lets the 16 or 32 MiB cache sit inside this part’s 128 MiB L2. PyTorch’s own kernel then measures 32.76  $\mu\text{s}$  and 49.67  $\mu\text{s}$  instead of 54.31 and 96.23: an inflation of 1.66 $\times$  and 1.94 $\times$ .

Defeating residency parity thus makes a production kernel look up to 1.94 $\times$  faster than it is. The artifact belongs to the measurement setup, not to our kernels, and it is large enough to manufacture or erase any fusion result in this report. We consider this the most useful methodological finding here.

#### C. Between-run variance

A fresh-process re-run of the attention grid, performed days after the committed run, reproduces all six cells within 1.1 %, with four of six inside 0.5 %. The reported fused-versus-graph gaps on Blackwell (16.13 and 8.55  $\mu\text{s}$ ) are an order of magnitude larger than this drift, which is what licenses the unlocked-clock caveat on that machine.

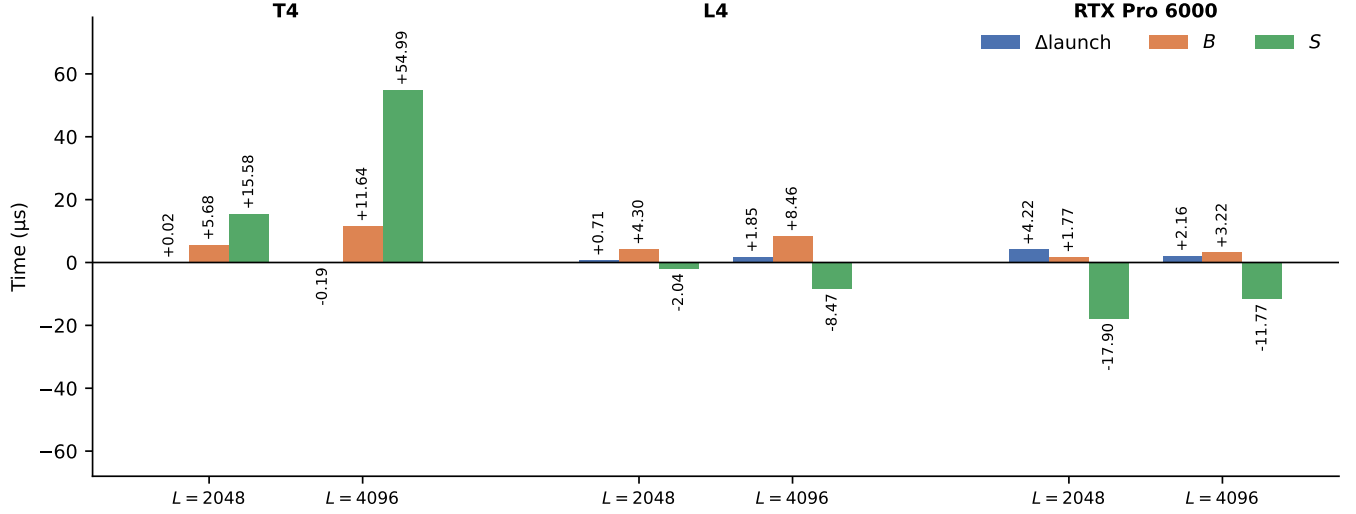


Fig. 3. Decomposition of the attention chain per GPU.  $\Delta_{\text{launch}}$  is what CUDA Graphs recover;  $B$  and  $S$  are what fusion adds on top.  $S$  is positive only on T4, and  $\Delta_{\text{launch}}$  grows with newer architectures.

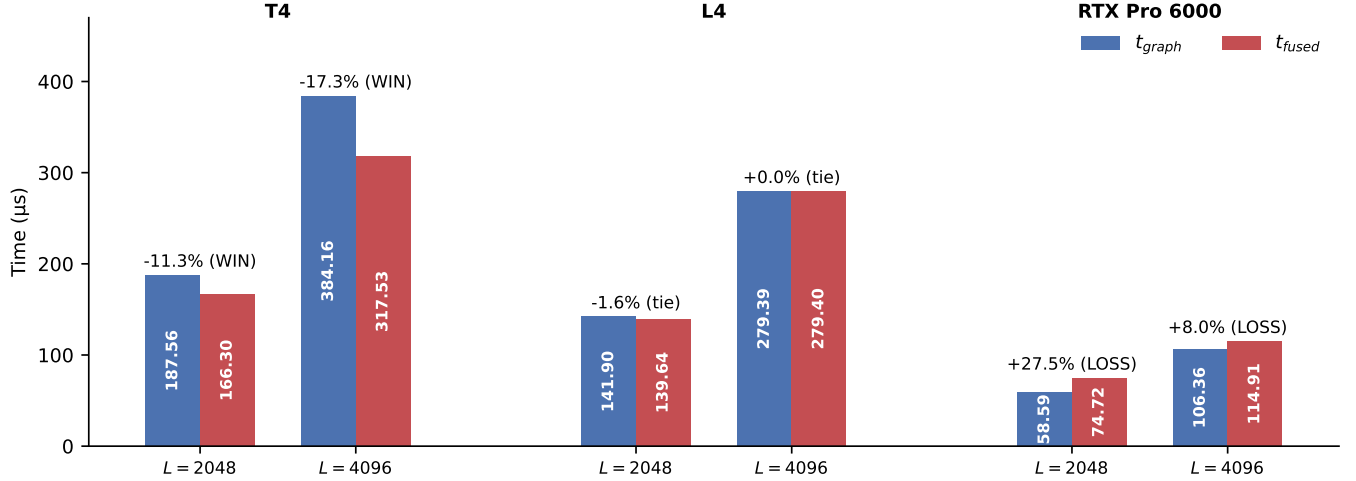


Fig. 4. The fused kernel against the graph baseline: a win on Turing, a tie on Ada, and a loss on Blackwell, with the protocol held fixed throughout.

TABLE V  
BLACKWELL SPLIT-COUNT SWEEP OF THE FUSED ATTENTION KERNEL, IN  $\mu\text{s}$ , WITH THE PERCENTAGE AGAINST THE CUDA-GRAPH BASELINE. BEST VALUE PER COLUMN IN BOLD.

| splits | $L=2048$     | vs graph       | $L=4096$      | vs graph      |
|--------|--------------|----------------|---------------|---------------|
| 1      | 463.36       | +699 %         | 914.02        | +758 %        |
| 2      | 231.43       | +299 %         | 452.82        | +325 %        |
| 4      | 120.96       | +109 %         | 231.90        | +118 %        |
| 8      | <b>69.53</b> | <b>+19.9 %</b> | 126.50        | +18.8 %       |
| 16     | 75.00        | +29.3 %        | 143.15        | +34.4 %       |
| 32     | 75.16        | +29.6 %        | <b>115.18</b> | <b>+8.1 %</b> |

The best configuration at each length is still slower than the graph baseline, by 19.9 % and 8.1 %, and slower than SDPA by 28.0 % and 19.7 %. The frozen T4 constant turns out to be optimal at  $L = 4096$  and within 7 % of optimal at  $L = 2048$ , so the pre-registration discipline cost nothing here.

This narrows the limitation without removing it. We swept one axis of one kernel. A differently *structured* fused kernel (different tiling, vectorization, scheduling, or architecture-specific features) remains untested, so an equal-tuning-budget study is still future work.

## VII. INTERPRETATION

The one architecture-stable conclusion is that fusion should be judged against graph replay. Everything else in this study moved between generations.

Candidate explanations for the inversion of  $S$  are hypotheses, not findings. Larger caches and higher SM counts may ab-

### D. Does tuning rescue fusion?

The most load-bearing limitation of this study is that the split count of the fused attention kernel was tuned on T4 and then frozen. We swept the kernel's one exposed parameter across  $\{1, 2, 4, 8, 16, 32\}$  on Blackwell (Table V, Fig. 6).

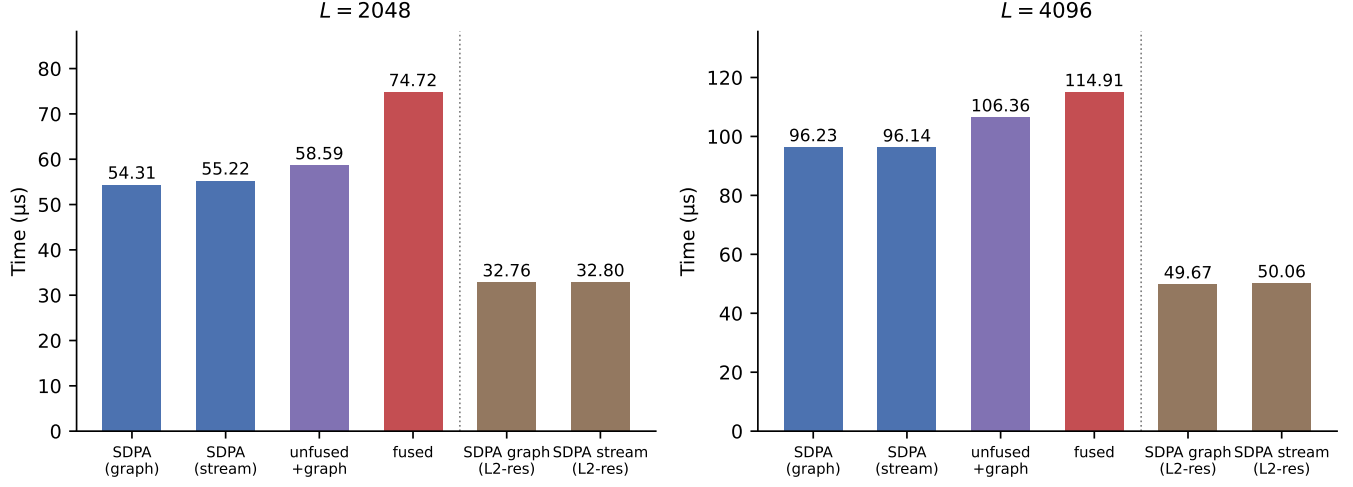


Fig. 5. Blackwell variants against the PyTorch SDPA reference point. The rightmost pair shows the same SDPA kernel measured with cache-residency parity defeated, which inflates it by 1.66–1.94 $\times$ .

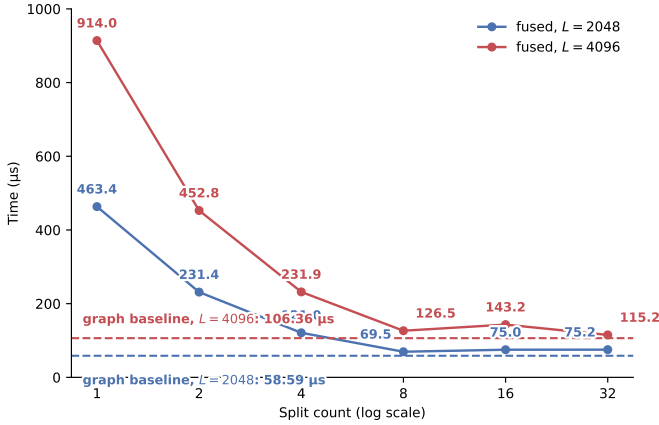


Fig. 6. Split-count sweep on Blackwell. Dashed lines mark the CUDA-Graph baseline at each context length; no configuration reaches it.

sorb the kernel-boundary drain that fusion removes on smaller parts; the split-KV decomposition adds recompute and partial-buffer traffic for which newer parts may pay relatively more; occupancy and register pressure differ across architectures. We assert none of these. The registered softmax-underutilization story predicted the wrong slope and cannot be the whole account. Identifying the mechanism would need controlled ablations (occupancy pinning, recompute isolation, cache-flush experiments) which we did not run.

The byte argument fails independently of the wall-clock outcomes. The eliminable fraction is about  $4/D$  by construction, and empirically the attention byte delta sat below DRAM counter resolution wherever counters were available. Decode-chain fusion at these sizes cannot be justified by byte elimination whatever the clock says. Because this follows from tensor shapes rather than from any particular kernel, it is the most transferable result here.

For practitioners the guidance is short. Enable CUDA Graphs first; that was the only intervention that helped on

every part. Before writing a fused kernel, compute  $B$  from tensor shapes using (3). If the eliminable fraction is a few percent, the byte argument cannot carry the decision, and the bet rests entirely on an architecture-specific structural effect whose sign this study shows can invert between generations.

## VIII. PRE-REGISTRATION AND ITS REFUTATIONS

Version 1 claimed that  $B$  accounted for at least 80 % of the attention gap. Our own gates refuted it on T4, where  $B$  explains roughly 20 %. A companion hypothesis, that the gap decomposes into  $\Delta_{\text{launch}} + B$  within 2 %, fell with it. A third, framed as “ $\Delta_{\text{launch}}$  explains the gap”, became undefined once residency parity made fused F1 and F2 slower than the graph baseline: there was no positive gap left to attribute. The refutation data is preserved in the artifact and motivated the  $B + S$  formulation.

Version 2 was registered before any confirmatory run, with gates frozen and every revision dated, including the indeterminate-reporting change described earlier, which moved no threshold and reduced our own reported pass counts.

The cache-residency artifact deserves explicit record. Before parity was enforced, fused F1 and F2 appeared to win; in fact the fused variants were reading cache-resident weights that the unfused variants streamed from DRAM. This is why only post-overhaul numbers appear here, and why Section VI-B exists.

## IX. RELATED WORK

The re-sweep of 2026-08-09 found no work that decomposes a fusion gain into launch and byte components against a CUDA-Graph baseline. That gap is what this report addresses.

*A note on scope of citation.* We cite only work whose bibliographic details we verified directly against arXiv or the publisher, and we prefer a short, checkable reference list to a long one. Several further systems in this area were surveyed during the re-sweep but are not cited individually, because they

neither change the positioning nor could be confirmed to our satisfaction.

**Attention kernels.** The FlashAttention lineage [14], [18] is IO-aware but targets training and prefill, where the arithmetic intensity is far higher than in batch-1 decode. FlashDecoding++ [13] and FlashInfer [19] address decode specifically; the split-KV structure of our fused variant follows the former. These are kernel designs, not measurements of where a fusion gain comes from.

**Fusion and megakernel systems.** MPK [20] compiles a whole model into a single persistent kernel; ClusterFusion [21] and its successor [22] widen the fusion scope to cluster-level collectives and then to a full transformer block. All report aggregate end-to-end speedups. None separates launch recovery from byte elimination, which is precisely the decomposition this report supplies.

**Serving systems as baselines, not competitors.** vLLM [1], SGLang [3], TensorRT-LLM [2], and torch.compile [23] establish CUDA-Graph replay as the production default; that default is what makes our baseline choice the correct one. The roofline model [11] is the ancestor of  $B$ , used here as a pre-implementation classifier rather than a post-hoc explanation.

**Independent evidence of architecture dependence.** Two recent results point the same way as ours from different directions. Chen [17] studies batch-1 decode on H100, A100, L40S, and L4 with a CUDA-Graphs A/B comparison, finding graphs worth  $1.259\times$  on H100 but only  $1.028\times$  on L4. That work fuses nothing and computes no byte estimate, so it does not close the gap identified above; but it corroborates the launch-side trend of Section V-E, and it supplies evidence on the two generations we could not access. It is not a substitute for the missing Ampere and Hopper fusion cells, because it never fuses anything. AutoMegaKernel [24] reports the complementary observation from the fusion side: its synthesized megakernels beat cuBLAS on inference-class parts (L4, L40S) but trail on training-class A100 and H100. Read together with our results, three independent studies now find that the value of both graph replay and fusion depends on which GPU you run it on, which is exactly why a single-architecture fusion result should not be generalized.

Finally, speculative decoding [25] reduces the *number* of decode steps rather than the per-step kernel cost, and is complementary to everything measured here.

## X. THREATS TO VALIDITY

**$S$  is a residual, not an identified mechanism.** Agreement in sign between two instruments shows which implementation is faster, never why.

**$B$  is a proportional estimate, not a physical byte cost or a bound.** No conclusion here rests on the magnitude of  $B$ ; the wall-clock sign carries both refutations.

**Fixed implementation, not fixed structure.** Section VI-D narrows this: tuning the split count does not rescue fusion on Blackwell. A differently structured fused kernel remains untested.

**The attention comparison is not a fusion-only ablation.** The fused path adds a split-KV decomposition and a reduction

kernel, so it differs from the unfused path in more than intermediate materialization.

**Narrow workload.** Three FP16 batch-1 chains at fixed geometry, with multi-head attention only: no grouped-query attention [26], paged KV [1], quantization, continuous batching, mixture-of-experts, or tensor parallelism.

**Three generations, not five.** Ampere and Hopper were not measured, because that hardware was unavailable. We state this as a boundary rather than a pending gap: the cross-GPU prediction those runs were registered to test was refuted at the second data point and inverted at the third, so they would add generational coverage rather than evidence.

**The Blackwell part is a MIG slice of a vGPU,** not a full die, and its host blocks performance counters, so that run is timing-only with unlocked clocks; Section VI-C bounds the resulting drift at 1.1 %.

**Baseline quality,** formerly a custom-versus-custom concern, is now anchored within 6.4–9.6 % of a production kernel (Section VI-A).

**Single toolchain.** CUDA 12.9 with drivers in the 580–595 range, and batch-1 decode only.

## XI. CONCLUSION

Three conclusions follow, in decreasing order of how strongly we would defend them.

First, CUDA-Graph replay is the only defensible baseline for a decode-fusion claim. This held on every architecture measured, and it is the one piece of guidance we would give unconditionally.

Second, byte elimination cannot justify decode-chain fusion at these sizes. The eliminable fraction is about  $4/D$  by construction, independent of context length, and sat below counter resolution wherever we could measure it. Because it follows from tensor shapes, this transfers beyond our kernels.

Third, fusion economics for short decode kernels are architecture-specific and do not extrapolate in either direction. A 17 % win on Turing became a tie on Ada and a 27 % loss on Blackwell, while the launch-bound chains moved the opposite way. Tuning did not rescue the loser, and a production kernel confirmed that the baseline was sound.

The methodology will outlast these particular numbers: report launch-versus-bytes over graphs, and register the prediction before running the confirmatory GPU. Individual fusion results are superseded monthly. What is still missing from the literature is a routine statement of *which term* was won.

## ARTIFACT AVAILABILITY

Kernels, harness, the frozen pre-registration, per-GPU raw CSVs with environment manifests, and one-command re-analysis are available at <https://github.com/animeshsri14/Decodebench>. The figures in this report are regenerated from the committed CSVs by `paper/make_figures.py`, so they cannot drift from the underlying data. Continuous integration runs CPU tests, a wheel build, and a compile-only CUDA matrix for sm\_75, sm\_89, and sm\_120.



## ACKNOWLEDGMENT

The authors thank the maintainers of the open-source inference stacks whose CUDA-Graph defaults motivated the central question of this report.

## REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [2] NVIDIA, “TensorRT-LLM: A TensorRT toolbox for optimized large language model inference,” <https://github.com/NVIDIA/TensorRT-LLM>, accessed 2026-08-09.
- [3] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2312.07104.
- [4] NVIDIA, “CUDA C++ programming guide, section 3.2.8: CUDA graphs,” <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, accessed 2026-08-09.
- [5] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [6] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, “The preregistration revolution,” *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.
- [7] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière *et al.*, “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [8] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [9] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *arXiv preprint arXiv:2104.09864*, 2021.
- [10] N. Shazeer, “GLU variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020.
- [11] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [12] NVIDIA, “Nsight Compute user guide,” <https://docs.nvidia.com/nsight-compute/>, accessed 2026-08-09.
- [13] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, Y. Dong *et al.*, “FlashDecoding++: Faster large language model inference on GPUs,” *arXiv preprint arXiv:2311.01282*, 2023.
- [14] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [15] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu *et al.*, “MLPerf inference benchmark,” in *Proc. ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- [16] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
- [17] J. Chen, “Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode,” *arXiv preprint arXiv:2605.30571*, 2026.
- [18] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [19] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, “FlashInfer: Efficient and customizable attention engine for LLM inference serving,” in *Proc. Machine Learning and Systems (MLSys)*, 2025, arXiv:2501.01005.
- [20] X. Cheng, Z. Zhang, Y. Zhou, J. Ji, J. Jiang, Z. Zhao, Z. Xiao, Z. Ye, Y. Huang, R. Lai, H. Jin, B. Hou, M. Wu, Y. Dong, X. Miao, T. Chen, and Z. Jia, “MPK: A compiler and runtime for mega-kernelizing tensor programs,” *arXiv preprint arXiv:2512.22219*, 2025.
- [21] X. Luo, Z. Liu, Y. Zhou, S. Fang, Z. Huang, Y. Feng, C. Zhang, S. Sun, Z. Zheng, J. Leng, and M. Guo, “ClusterFusion: Expanding operator fusion scope for LLM inference via cluster-level collective primitive,” *arXiv preprint arXiv:2508.18850*, 2025.
- [22] C. Jin, H. Yu, and X. Chen, “ClusterFusion++: Expanding cluster-level fusion to full transformer-block decoding,” *arXiv preprint arXiv:2604.23553*, 2026.
- [23] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao *et al.*, “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *Proc. 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [24] J. Jaber and O. Jaber, “AutoMegaKernel: A statically-checked agent harness for self-retargeting megakernel synthesis,” *arXiv preprint arXiv:2606.09682*, 2026.
- [25] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proc. 40th International Conference on Machine Learning (ICML)*, 2023.
- [26] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.